

Performance of the multi-RHS dielectric BIE solver: geometry, parallelism, and right-hand-side batching

BoundaryIntegral.jl / four_index_integral_solver

Abstract

We benchmark the FMM3D-accelerated dielectric boundary-integral solver used to assemble the four-index integrals V_{ijkl} of monolayer graphene. Three questions are addressed, each backed by the measurements in `../data/`: (i) how the thin-slab interface geometry compares with a 3D-uniform point cloud; (ii) the speedup actually obtained from OpenMP threads and from batching right-hand sides (`nd`); and (iii) the mechanism by which the multi-RHS formulation accelerates the solve. The headline results are: the slab geometry costs up to $7\times$ a uniform cloud at equal problem size; thread scaling saturates near 32–64 cores (~ 18 – $20\times$ at 96), while `nd` batching gives a flat $\sim 3.7\times$, the two compounding to $\sim 73\times$; and the full block solve inherits the FMM’s $\sim 4\times$ per-RHS amortization, which we trace—from the FMM3D Fortran and a direct phase breakdown—to per-interaction geometry and translation operators being computed once and reused across all densities, *not* to tree construction.

1 Setup and methodology

All data are from the monolayer graphene model (cRPA data `k_323201_nb_144_c_15`): a dielectric slab of lateral size $L = 90$, thickness $L_z = 3.35$ (bohr), $\epsilon_{\text{in}} = 3.5$, discretized by panel quadrature on the box faces. The FMM kernel is Laplace `lfmm3d` from FMM3D. Runs are on Flatiron `genoa` nodes (96 cores, 1.5 TB); every thread configuration ran on its *own exclusive node* (Slurm array, one node per `OMP_NUM_THREADS`) so the timings are free of core-sharing contention. Threads were pinned by setting *both* `OMP_NUM_THREADS` and `OPENBLAS_NUM_THREADS`.

Table 1: Benchmarks and their parameters. N = number of interface unknowns (slab) or random points (uniform), set by the edge-refinement level. Data files under `../data/`.

benchmark (script)	system	N	<code>nd</code> / K
FMM kernel (<code>bench_fmm.jl</code>)	slab interface, far source	33 696 / 93 312 / 215 136 / 461 376	1,4,16,36
uniform baseline (<code>bench_fmm_uniform.jl</code>)	3D-uniform points	same N	same
solve breakdown (<code>bench_solve_breakdown.jl</code>)	2-shell graphene	219 456	block 1, 3
per-RHS solve (<code>bench_per_rhs.jl</code>)	2-shell graphene	219 456	block 1, 3
phase breakdown (<code>tools/nd_timing.f90</code>)	3D-uniform	200 000	1, 36

Throughput is reported as $\Theta = N \text{nd} / t_{\text{fmm}}$ in million source-RHS evaluations per second. The FMM cost is modeled as $t_{\text{fmm}}(\text{nd}) \approx a + b \text{nd}$, with a the `nd`-independent (shared) work and b the per-density (marginal) work; the per-RHS amortization ceiling is then $(a + b)/b$.

2 Uniform versus non-uniform point distributions

To isolate the effect of geometry, we ran the identical FMM kernel (`nd`, N , threads, tolerance) on (a) the real slab interface and (b) a 3D-uniform random cloud of the same N (`bench_fmm.jl` vs `bench_fmm_uniform.jl`). Figure 1 shows the throughput along each of the three knobs; Figure 2 shows the full threads $\times$ N throughput maps and their ratio.

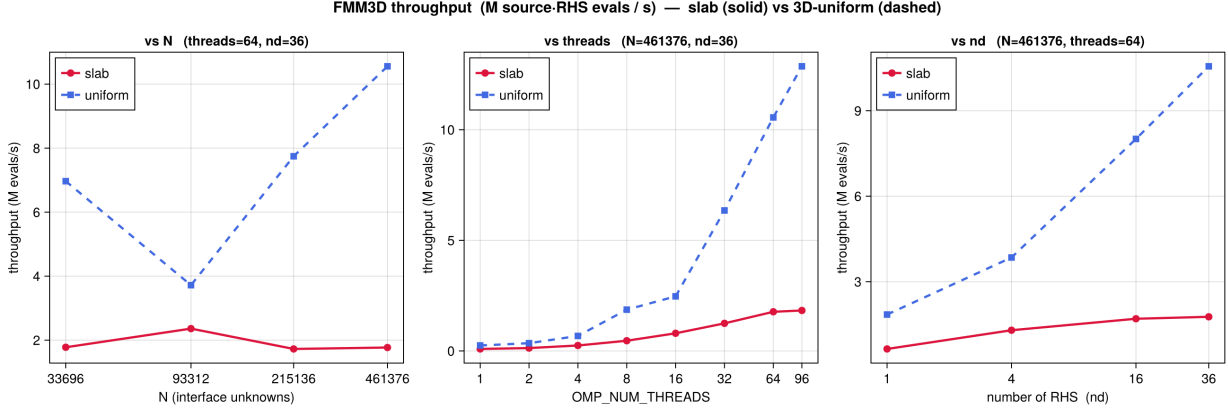


Figure 1: FMM3D throughput vs. N , threads, and nd (slab solid red, uniform dashed blue), each panel varying one knob with the others fixed (threads= 64, nd = 36, N = 461 376). Uniform dominates the slab in every panel. Data: `../data/fmm_bench.csv`, `fmm_bench_uniform.csv`.

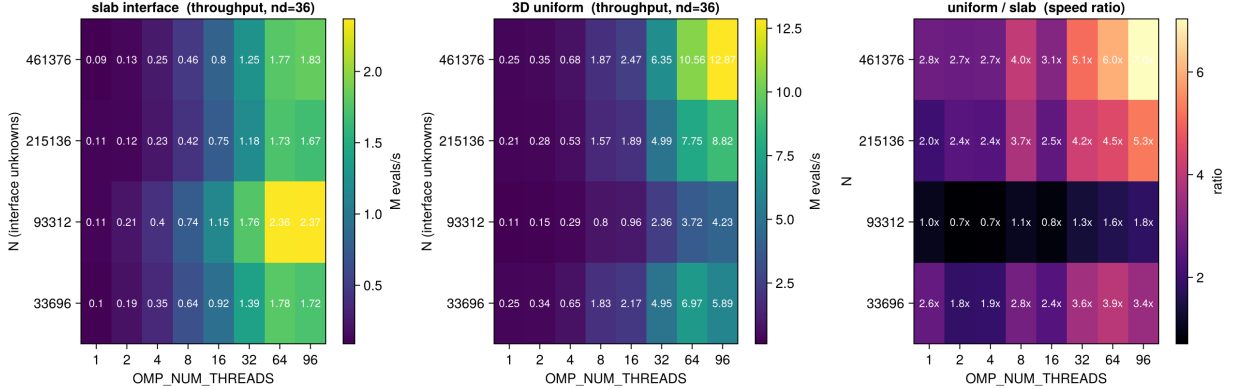


Figure 2: Throughput (M evals/s, nd = 36) over threads $\times$ N : slab (left), uniform (centre), ratio uniform/slab (right).

The slab is up to 7 $\times$ slower. Peak throughput is $\Theta_{\max}^{\text{slab}} = 2.37$ (at 96 threads, N = 93 312) versus $\Theta_{\max}^{\text{uni}} = 12.87$ (96 threads, N = 461 376). Head-to-head at the largest size the penalty grows with thread count (Table 2): from 2.75 $\times$ single-threaded to 7.03 $\times$ at 96 threads (N = 461 376).

The penalty factors into “more work” $\times$ “worse scaling”. At N = 461 376,

$$\underbrace{7.03\times}_{96 \text{ threads}} = \underbrace{2.75\times}_{\text{single-threaded}} \times \underbrace{2.54\times}_{\text{scaling gap}}, \quad (1)$$

Table 2: Slab/uniform ratio of t_{fmm} at $\text{nd} = 36$ (how many times slower the slab is). The $N = 93\,312$ uniform column (< 1 , in parentheses) is anomalous—see text.

threads	$N = 33\,696$	$N = 93\,312$	$N = 215\,136$	$N = 461\,376$
1	2.61	(1.04)	1.99	2.75
8	2.85	(1.07)	3.70	4.04
32	3.56	(1.35)	4.23	5.09
64	3.92	(1.58)	4.49	5.96
96	3.42	(1.79)	5.27	7.03

where the single-thread factor is the raw extra work and the scaling gap is the ratio of parallel speedups: the slab scales $1 \rightarrow 96$ threads by only $20.4\times$, the uniform cloud by $52.0\times$ (both at $\text{nd} = 36$, $N = 461\,376$, from `fmm_bench.csv` / `fmm_bench_uniform.csv`). The cause is the octree: FMM3D’s root box is a cube, but the points occupy only $L_z/L = 3.35/90 \approx 3.7\%$ of its z -extent, so the first $\sim \log_2(90/3.35) \approx 5$ subdivision levels split empty space (the tree refines like a 2D quadtree, branching ~ 4 not 8) and must go deeper to reach the same leaf occupancy; the resulting boxes are clustered in a thin sheet, which also load-balances poorly across threads.

Caveat. The uniform $N = 93\,312$ point is anomalous: its throughput is non-monotonic in N (lower than both $N = 33\,696$ and $215\,136$ at every thread count), giving a spurious ratio < 1 in Table 2. This is an unlucky seeded random-point tree and should be re-run with a different seed before being quoted.

3 Speedup from threads and right-hand-side batching

Threads and nd are two orthogonal levers on the FMM kernel (Fig. 3, 4). We report both relative to the slab kernel, since that is the production geometry.

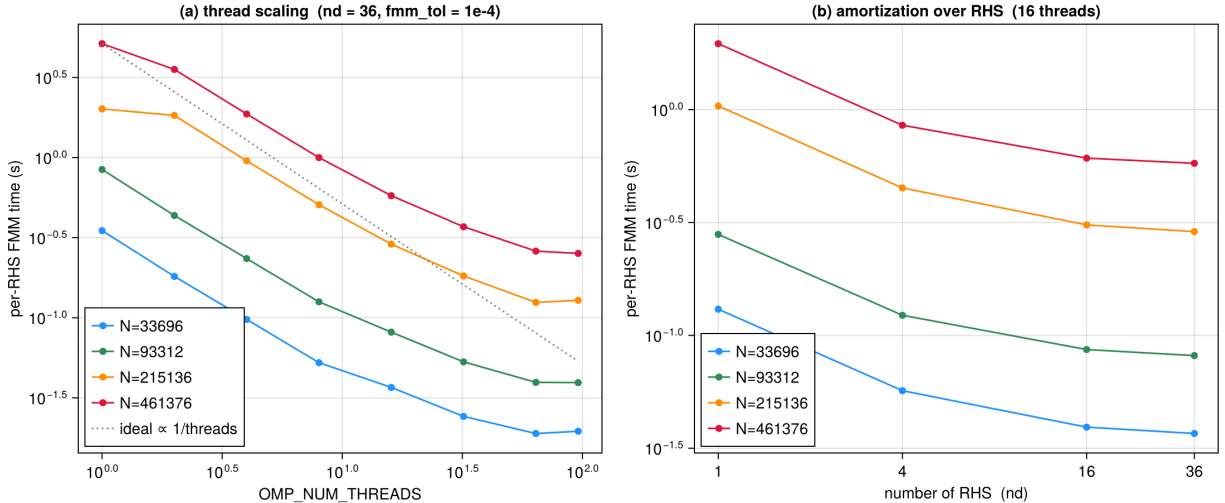


Figure 3: Per-RHS FMM time (slab). (a) thread scaling at $\text{nd} = 36$: near-ideal to ~ 16 threads, then saturating. (b) amortization over nd at 16 threads: falls and plateaus. Data: `../data/fmm_bench.csv`.

Batching nd: a flat $\sim 3.7\times$ that plateaus. At fixed threads the per-RHS FMM time t_{fmm}/nd falls and saturates (Table 3, $N = 461\,376$, 1 thread): the factor is $3.72\times$ at $\text{nd} = 36$ and essentially independent of N and thread count. It cannot exceed $\sim 4\times$ here; §4 explains why.

Table 3: nd amortization of the FMM kernel (slab, $N = 461\,376$, 1 thread).

nd	total t_{fmm} (s)	per-RHS (s)	amortization vs. nd=1
1	19.10	19.10	$1.00\times$
4	31.76	7.94	$2.41\times$
16	87.46	5.47	$3.49\times$
36	185.03	5.14	$3.72\times$

Threads: $\sim 18\text{--}20\times$, saturating at $32\text{--}64$. Per-RHS FMM time vs. threads (Table 4) is near-ideal ($\propto 1/\text{threads}$) up to ~ 16 threads, then saturates; **96 threads is no better than 64** (and slightly worse for small N). The $1 \rightarrow 96$ speedup grows with problem size: $17.9\times$ at $N = 33\,696$ up to $20.4\times$ at $N = 461\,376$.

Table 4: Per-RHS FMM time (s), slab, $\text{nd} = 36$. Bottom row: $1 \rightarrow 96$ speedup.

threads	$N = 33\,696$	$N = 93\,312$	$N = 215\,136$	$N = 461\,376$
1	0.350	0.843	2.013	5.140
16	0.037	0.081	0.288	0.578
64	0.019	0.040	0.125	0.261
96	0.020	0.039	0.129	0.252
$1 \rightarrow 96$	$17.9\times$	$21.4\times$	$15.7\times$	$20.4\times$

Combined: up to $\sim 73\times$. Folding both levers, the per-RHS speedup relative to the ($\text{nd}=1$, 1 thread) baseline, $36t(1,1)/t(36,T)$, is given in Table 5 and visualized in Figure 4. The robust operating point is $\text{nd} = 36$ at 64 threads ($73\times$); 96 threads ($76\times$) is within noise. The combined factor is approximately the product ($3.7\times$ from nd) $\times$ (thread factor), with the nd part constant and the thread part saturating.

Table 5: Combined per-RHS speedup vs. ($\text{nd}=1$, 1 thread), slab, $N = 461\,376$.

threads	1	8	16	64	96
speedup	$3.7\times$	$19.1\times$	$33.0\times$	$73.3\times$	$75.7\times$

These are *kernel* speedups and must not be conflated: $\sim 3.7\times$ is nd -batching at fixed threads; $\sim 18\text{--}20\times$ is threads $1 \rightarrow 96$; $\sim 73\times$ is both together.

4 How multi-RHS batching accelerates the solve

For one orbital centre i , the neighbour pair densities $\rho_{ij} = \varphi_i \varphi_j$ are all right-hand sides of the *same* operator $A = \frac{1}{2}\gamma^{-1} + D^\top$. They are therefore solved together by block GMRES, each matrix-vector product being a single $\text{nd} = K$ batched FMM. Two layers of acceleration follow.

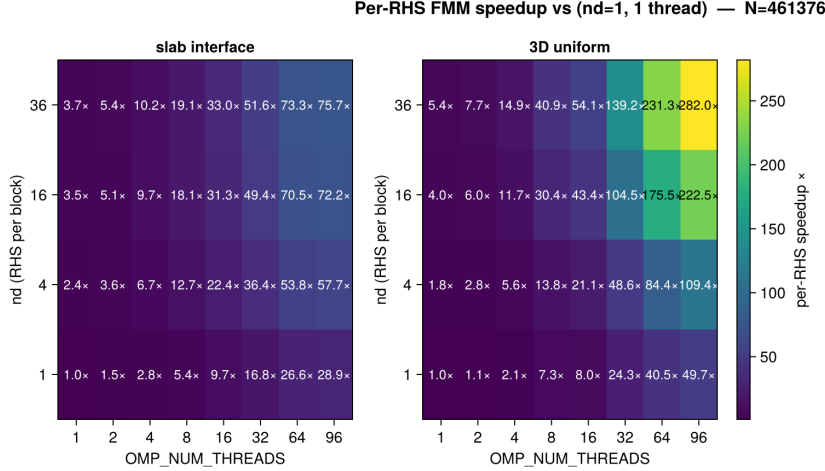


Figure 4: Per-RHS FMM speedup over (nd, threads) vs. the (nd=1, 1 thr) baseline at $N = 461\,376$: slab (left) vs. uniform (right). The slab corner reaches $75.7\times$, the uniform $282\times$ —the geometry penalty of §2 again, now in the thread axis (uniform scales far better).

4.1 FMM level: why batched densities are cheap

A single `lfmm3d` call is *not* dominated by tree construction. Calling `lfmm3d_ndiv` directly and reading its returned six-phase `timeinfo` (`tools/nd_timing.f90`; $N = 200\,000$ uniform, 1 thread) gives Table 6.

Table 6: FMM phase breakdown (s) at $\text{nd} = 1$ and $\text{nd} = 36$, with the $a + b\text{nd}$ split ($b = (t_{36} - t_1)/35$). Data: `../data/fmm_phase_breakdown.csv`.

phase	nd=1	nd=36	shared a	per-RHS b
P2M (form multipoles)	0.179	0.833	0.16	0.019
M2M (upward)	0.178	2.167	0.12	0.057
M2L (mp→local)	2.036	15.943	1.64	0.40
L2L (downward)	0.166	2.082	0.11	0.055
L2P (eval locals)	0.214	0.624	0.20	0.012
P2P (direct/near)	1.129	4.954	1.02	0.11
tree + setup	0.088	0.438	—	—
total	3.99	27.04	≈ 3.34 (84%)	≈ 0.65 (16%)

Tree construction is only $\sim 0.09\text{ s}$, $\approx 2\%$ of the call (the “tree+setup” row grows with nd solely because it includes zeroing the nd -times-larger expansion workspace, not tree building). The shared $\sim 84\%$ is **per-interaction geometry and translation operators**, dominated by M2L (1.64 s) and the near-field P2P (1.02 s). The FMM3D Fortran shows directly why this is reused across all densities: every kernel computes the geometry once and loops over nd for cheap multiply-adds. For the direct (near) kernel `l3ddirectcg`:

```
cd = inv4pi/sqrt(dd) ! 1/r : the expensive sqrt, ONCE per source-target pair
cd1 = -cd/dd
do idim = 1,nd ! inner loop over densities: only FMAs
  pot(idim,i) = pot(idim,i) + cd *charge(idim,j)
  grad(idim,1,i) = grad(idim,1,i) + cd1*zdiff(1)*charge(idim,j)
```

```
...
enddo
```

and likewise in the far field the associated-Legendre recurrences (`ylgndrfrw`) and the M2L operators are formed once per box and reused inside a `do idim=1,nd` loop (`l3dmpmp`, `l3dmpeval`). Hence $t_{\text{fmm}}(\text{nd}=36) \approx 6.8 t_{\text{fmm}}(\text{nd}=1)$ here (not $36\times$), and the per-RHS ceiling is $(a+b)/b \approx 6\times$ for uniform; the slab’s heavier near-field and deeper tree lower the shared fraction, giving the $\sim 3.7\text{--}4\times$ of Table 3.

4.2 Solve level: block GMRES inherits the FMM’s $\sim 4\times$

Instrumenting the block solve (`bench_solve_breakdown.jl`, $N = 219456$, threads pinned) gives Table 7. Two facts pin down the mechanism:

Table 7: Block-solve cost breakdown (s); `nmv=operator` applications. Data: `../data/solve_breakdown.csv`.

OMP	K	iter	nmv	total	FMM	matmul	glue	GMRES	per-RHS
1	1	7	7	61.4	57.9 (94%)	0.015	0.70	2.83	61.4
1	36	7	7	552.6	537.4 (97%)	0.232	12.7	2.25	15.3
16	1	7	7	11.6	8.1 (70%)	0.010	0.65	2.85	11.6
16	36	7	7	91.6	77.6 (85%)	0.230	12.7	1.06	2.54

1. **The FMM dominates the solve** (94–97% at 1 thread, 70–85% at 16); the near-correction `matmul` is negligible ($< 0.5\%$) and GMRES is a small fixed overhead ($\sim 1\text{--}3\text{ s}$).
2. **Block GMRES does not save iterations:** `iter=nmv=7` for both $K = 1$ and $K = 36$. It applies the operator the same number of times regardless of K ; the sole benefit is that each application is one `nd = K` batched FMM.

Consequently the per-RHS *solve* amortization is $61.4/15.3 = 4.0\times$ (1 thread) and $11.6/2.54 = 4.55\times$ (16 threads)—it simply *tracks the FMM* (whose own `nd` factor is $537.4/57.9 = 9.3\times$ for 36 densities, i.e. $3.9\times$ per RHS). Sweeping K (`bench_per_rhs.jl`, 16 threads) confirms the per-RHS solve time *plateaus* at the FMM ceiling rather than falling as $1/K$ (Fig. 5): from 15.63 s ($K = 1$) to 3.62 s ($K = 36$), a $4.3\times$ amortization.

Methodological note. An earlier unpinned run of `bench_per_rhs.jl` reported a spurious $\sim 18\times$ “per-RHS speedup tracking $1/K$ ”. With threads unpinned, its small- K solves ran effectively single-threaded while its large- K solves picked up OpenMP/BLAS threads, folding a $\sim 4\times$ thread speedup onto the K axis. The arithmetic is decisive: at 1 thread the $K = 36$ FMM alone is 537 s (Table 7), so the 127 s total that run reported for $K = 36$ is impossible single-threaded. The honest number is $\sim 4\times$; threads and `nd` are separate levers and must be measured with both `OMP_NUM_THREADS` and `OPENBLAS_NUM_THREADS` pinned.

5 Summary and recommendations

- **Geometry** (§2): the thin slab interface costs up to $7\times$ a uniform cloud of equal N — $2.75\times$ in raw work (cubic octree wasting the slab’s thin dimension) and $2.5\times$ in worse thread scaling. This, not the kernel or tolerance, is the dominant inefficiency.

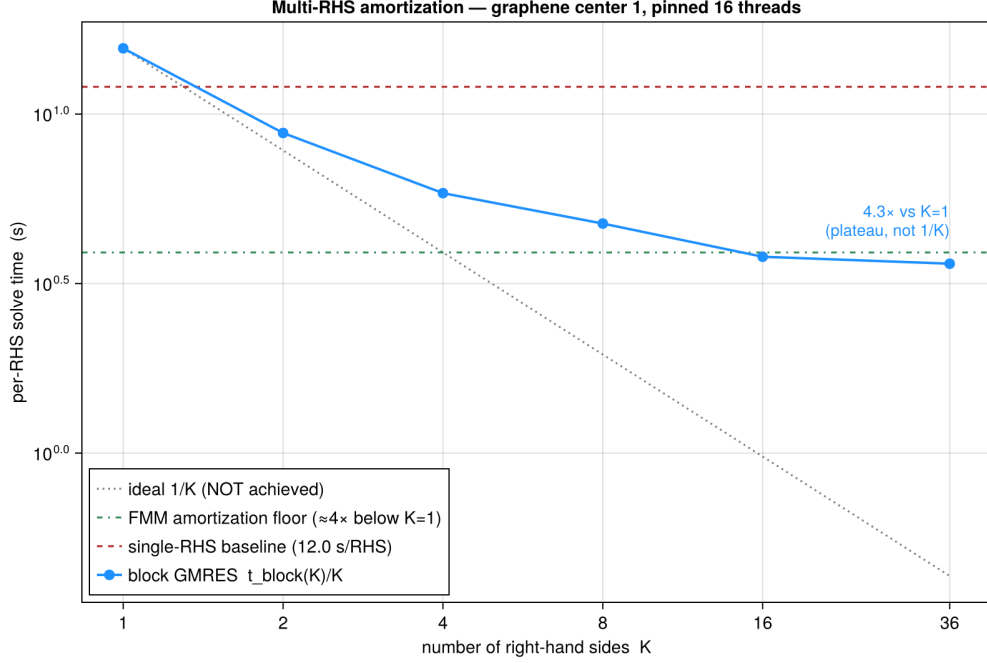


Figure 5: Per-RHS block-solve time vs. K (pinned 16 threads). The data plateau at the FMM amortization floor ($\sim 4\times$ below $K=1$), *not* along the dotted ideal $1/K$ line. Data: `../data/per_rhs.csv`.

- **Threads** (§3): run at ~ 32 – 64 cores; the FMM saturates and 96 threads buys nothing (~ 18 – $20\times$ over a single thread at large N).
- **Batching** (§3, 4): always solve co-located densities as a block—a free ~ 3.7 – $4\times$ per RHS. Combined with threads this is $\sim 73\times$.
- **Mechanism** (§4): the multi-RHS win comes entirely from the batched FMM, where $\sim 84\%$ of a call (M2L operators, the $1/r$ near field, Legendre recurrences) is computed once and reused across all densities; tree construction is $\sim 2\%$. Block GMRES adds no iteration savings.
- **Open items:** thread the single-threaded “glue” (charge assembly + $\nabla \cdot n$ contraction), which reaches $\sim 14\%$ of the solve at 16 threads and grows with core count; re-run the anomalous uniform $N = 93312$ point; an octree matched to the near-2D distribution is where the remaining geometry factor lives.